python™

WsCube Tech®
Upskilling Bharat

Banking System

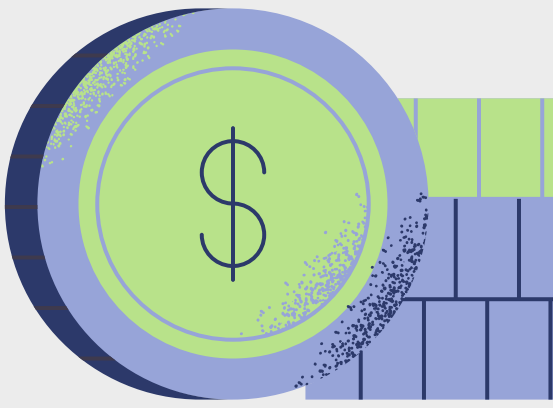
Python Bank Account Management System

PRESENTED BY: **DIVYA SHARMA**

Introduction

- Developed a Python application to manage bank accounts and perform basic banking operations.
- Implemented features like account creation, deposits, withdrawals, and fund transfers.
- Used file handling to store and retrieve account and transaction data.
- Built a menu-driven system to simulate real-world banking workflows and financial tracking.

Key Features



01 ACCOUNT CREATION AND MANAGEMENT

02 DEPOSIT, WITHDRAWAL, AND TRANSFER FUNCTIONALITY

03 TRANSACTION HISTORY TRACKING

04 DATA PERSISTENCE USING FILE HANDLING

05 INPUT VALIDATION AND ERROR HANDLING

What are we building?

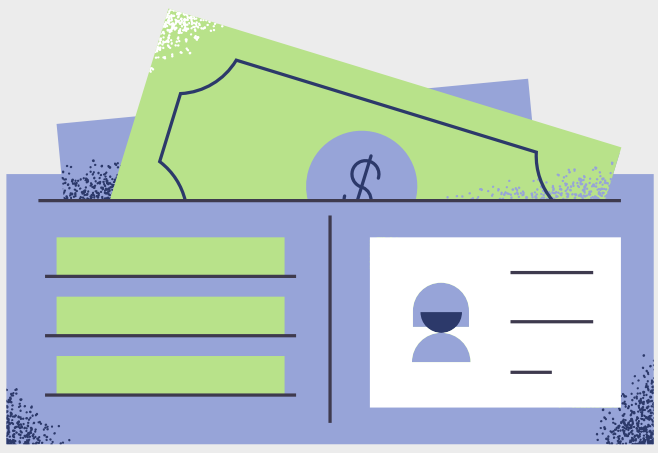
A MINI BANKING SYSTEM THAT CAN:

- Create accounts
- Store customer data
- Perform transactions
- Save data permanently
- Generate reports

Real-world analogy

A BANK SYSTEM MUST:

BANKING NEED	PROGRAMMING SOLUTION
STORE CUSTOMERS	DICTIONARY
SAVE RECORDS	FILE STORAGE
TRACK TRANSACTIONS	LIST HISTORY
PREVENT ERRORS	VALIDATION RULES



Python Bank Account Management System!

Python Code--

```python
"""
Indian Banking System
"""
import pickle
import os
import numpy as np
from datetime import datetime

# 1.
DATA_FILE = "easy_bank.dat"
accounts = {}

def load_data():
    global accounts
    if os.path.exists(DATA_FILE):
        with open(DATA_FILE, "rb") as f:
            accounts = pickle.load(f)

def save_data():
    with open(DATA_FILE, "wb") as f:
        pickle.dump(accounts, f)
    print("Data saved successfully!")

# 2.
def create_account():
    name = input("Enter Name: ")
    # Bonus: Login Feature
    pin = input("Set a 4-digit PIN: ")
    print("1. Savings  2. Current")
    acc_type = "Savings" if input("Choice: ") == '1' else "Current"
```

```python
    # Input Validation
    try:
        balance = float(input("Initial Deposit: "))
        if balance < 0: raise ValueError
    except:
        print("Invalid amount!")
        return

    # Auto-generate Account Number
    acc_num = 100001 if not accounts else max(accounts.keys()) + 1

    # Store in data structure
    accounts[acc_num] = {
        "name": name, "pin": pin, "type": acc_type,
        "balance": balance, "history": [balance]
    }
    print(f"Account Created! Your Account Number is: {acc_num}")

# 3.
def perform_transaction():
    try:
        acc_num = int(input("Enter Account Number: "))
        if acc_num not in accounts:
            print("Account not found!")
            return

        # Bonus: Security check
        if input("Enter PIN: ") != accounts[acc_num]["pin"]:
            print("Wrong PIN!")
            return
        print("1. Deposit  2. Withdraw  3. Transfer")
        choice = input("Choice: ")

        if choice == '1':
            amt = float(input("Amount: "))
            accounts[acc_num]["balance"] += amt
            accounts[acc_num]["history"].append(amt)
        elif choice == '2':
            amt = float(input("Amount: "))
            # Check balance
            if amt <= accounts[acc_num]["balance"]:
                accounts[acc_num]["balance"] -= amt
                accounts[acc_num]["history"].append(-amt) # Negative for wi
            else:
                print("Insufficient funds!")
        elif choice == '3':
            target = int(input("Recipient Account No: "))
            amt = float(input("Amount: "))
            if target in accounts and amt <= accounts[acc_num]["balance"]:
                accounts[acc_num]["balance"] -= amt
                accounts[target]["balance"] += amt
                accounts[acc_num]["history"].append(-amt)
                accounts[target]["history"].append(amt)
                print("Transfer Successful!")
    except:
        print("Error in transaction.")
```

```python
# 4.
def show_report():
    acc_num = int(input("Enter Account Number: "))
    if acc_num in accounts:
        acc = accounts[acc_num]
        print(f"Holder: {acc['name']} | Balance: ₹{acc['balance']}")

        # NumPy Statistics
        txns = np.array(acc["history"])
        print(f"Total Transactions: {len(txns)}")
        print(f"Average Transaction: {np.mean(np.abs(txns)):.2f}")
    else:
        print("Account not found.")

# 5.
def main():
    load_data()
    while True:
        print("\n--- BANKING MENU ---")
        print(["1. New Account  2. Transaction  3. Report  4. Exit"])
        choice = input("Select: ")

        if choice == '1': create_account()
        elif choice == '2': perform_transaction()
        elif choice == '3': show_report()
        elif choice == '4': save_data(); break
        else: print("Invalid choice.")

if __name__ == "__main__":
    main()
```

Output

1. USER INTERFACE (MAIN MENU OUTPUT)

```
Indian Banking System


--- BANKING MENU ---
['1. New Account  2. Transaction  3. Report  4. Exit']
Select:
```

2. OPTION 1 → NEW ACCOUNT CREATION

```
Indian Banking System


--- BANKING MENU ---
['1. New Account  2. Transaction  3. Report  4. Exit']
Select: 1
Enter Name: Divya
Set a 4-digit PIN: 12345
1. Savings  2. Current
Choice: 1
Initial Deposit: 55000
Account Created! Your Account Number is: 100002
```

3. OPTION 2 → TRANSACTION MENU

CASE A: DEPOSIT

```
Indian Banking System


--- BANKING MENU ---
['1. New Account  2. Transaction  3. Report  4. Exit']
Select: 2
Enter Account Number: 100002
Enter PIN: 12345
1. Deposit  2. Withdraw  3. Transfer
Choice: 1
Amount: 500000
```

CASE B: WITHDRAW

```
Indian Banking System

--- BANKING MENU ---
['1. New Account  2. Transaction  3. Report  4. Exit']
Select: 2
Enter Account Number: 100002
Enter PIN: 12345
1. Deposit  2. Withdraw  3. Transfer
Choice: 2
Amount: 10000
```

CASE C: TRANSFER

```
Indian Banking System

--- BANKING MENU ---
['1. New Account  2. Transaction  3. Report  4. Exit']
Select: 2
Enter Account Number: 100002
Enter PIN: 12345
1. Deposit  2. Withdraw  3. Transfer
Choice: 3
Recipient Account No: 100001
Amount: 15000
Transfer Successful!
```

4. OPTION 3 → ACCOUNT REPORT

```
Indian Banking System


--- BANKING MENU ---
['1. New Account  2. Transaction  3. Report  4. Exit']
Select: 3
Enter Account Number: 100002
Holder: Divya | Balance: ₹530000.0
Total Transactions: 4
Average Transaction: 145000.00
```

5. OPTION 4 → EXIT

```
Indian Banking System


--- BANKING MENU ---
['1. New Account  2. Transaction  3. Report  4. Exit']
Select: 4
Data saved successfully!
```

Banking System – Key Functional Capabilities

- Loads and saves account data using file handling (pickle).
- Creates new accounts with auto-generated account numbers.
- Implements PIN-based authentication for security.
- Supports deposit, withdrawal, and transfer operations.
- Validates balance before processing withdrawals/transfers.
- Stores transaction history for each account.
- Generates simple reports with balance and average transaction using numpy.
- Runs through a menu-driven interface for user interaction.

Python Concepts & Topics Used

- VARIABLES AND DATA TYPES (INT, FLOAT, STRING, DICTIONARY, LIST)
- DICTIONARIES FOR STRUCTURED DATA STORAGE
- FUNCTIONS FOR MODULAR PROGRAMMING
- CONDITIONAL STATEMENTS (IF-ELIF-ELSE)
- LOOPS (WHILE LOOP FOR MENU SYSTEM)
- EXCEPTION HANDLING (TRY-EXCEPT)
- FILE HANDLING (READ/WRITE IN BINARY MODE)
- PICKLE MODULE FOR OBJECT SERIALIZATION
- NUMPY FOR BASIC STATISTICAL CALCULATIONS
- USER INPUT HANDLING (INPUT FUNCTION)
- GLOBAL VARIABLES AND SCOPE MANAGEMENT
- BASIC PROGRAM STRUCTURE (IF __NAME__ == "__MAIN__":)

python™

THANK YOU

LinkedIn	www.linkedin.com/in/divya9916
GitHub	github.com/Divya9916